In [6]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
```

In [7]:
```
df = pd.read_csv('Mall_Customers.csv')
print("Dataset Overview:")
print(df.head())
print(f"\nDataset shape: {df.shape}")
print(df.describe())
```

```
Dataset Overview:
   CustomerID   Genre  Age  Annual Income (k$)  Spending Score (1-100)
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40

Dataset shape: (200, 5)
       CustomerID         Age  Annual Income (k$)  Spending Score (1-100)
count  200.000000  200.000000          200.000000              200.000000
mean   100.500000   38.850000           60.560000               50.200000
std     57.879185   13.969007           26.264721               25.823522
min      1.000000   18.000000           15.000000                1.000000
25%     50.750000   28.750000           41.500000               34.750000
50%    100.500000   36.000000           61.500000               50.000000
75%    150.250000   49.000000           78.000000               73.000000
max    200.000000   70.000000          137.000000               99.000000
```

In [8]:
```
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]
```

In [9]:
```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled
```

Out[9]:
```
array([[-1.73899919, -0.43480148],
       [-1.73899919,  1.19570407],
       [-1.70082976, -1.71591298],
       [-1.70082976,  1.04041783],
       [-1.66266033, -0.39597992],
       [-1.66266033,  1.00159627],
       [-1.62449091, -1.71591298],
       [-1.62449091,  1.70038436],
       [-1.58632148, -1.83237767],
       [-1.58632148,  0.84631002],
       [-1.58632148, -1.4053405 ],
       [-1.58632148,  1.89449216],
       [-1.54815205, -1.36651894],
       [-1.54815205,  1.04041783],
       [-1.54815205, -1.44416206],
       [-1.54815205,  1.11806095],
       [-1.50998262, -0.59008772],
       [-1.50998262,  0.61338066],
       [-1.43364376, -0.82301709],
       [-1.43364376,  1.8556706 ],
       [-1.39547433, -0.59008772],
       [-1.39547433,  0.88513158],
       [-1.3573049 , -1.75473454],
       [-1.3573049 ,  0.88513158],
       [-1.24279661, -1.4053405 ],
       [-1.24279661,  1.23452563],
       [-1.24279661, -0.7065524 ],
       [-1.24279661,  0.41927286],
       [-1.20462718, -0.74537397],
       [-1.20462718,  1.42863343],
       [-1.16645776, -1.7935561 ],
       [-1.16645776,  0.88513158],
       [-1.05194947, -1.7935561 ],
       [-1.05194947,  1.62274124],
       [-1.05194947, -1.4053405 ],
       [-1.05194947,  1.19570407],
       [-1.01378004, -1.28887582],
       [-1.01378004,  0.88513158],
       [-0.89927175, -0.93948177],
       [-0.89927175,  0.96277471],
       [-0.86110232, -0.59008772],
       [-0.86110232,  1.62274124],
       [-0.82293289, -0.55126616],
       [-0.82293289,  0.41927286],
       [-0.82293289, -0.86183865],
       [-0.82293289,  0.5745591 ],
       [-0.78476346,  0.18634349],
       [-0.78476346, -0.12422899],
       [-0.78476346, -0.3183368 ],
       [-0.78476346, -0.3183368 ],
       [-0.70842461,  0.06987881],
       [-0.70842461,  0.38045129],
       [-0.67025518,  0.14752193],
       [-0.67025518,  0.38045129],
       [-0.67025518, -0.20187212],
       [-0.67025518, -0.35715836],
       [-0.63208575, -0.00776431],
       [-0.63208575, -0.16305055],
       [-0.55574689,  0.03105725],
       [-0.55574689, -0.16305055],
       [-0.55574689,  0.22516505],
       [-0.55574689,  0.18634349],
       [-0.51757746,  0.06987881],
       [-0.51757746,  0.34162973],
       [-0.47940803,  0.03105725],
```

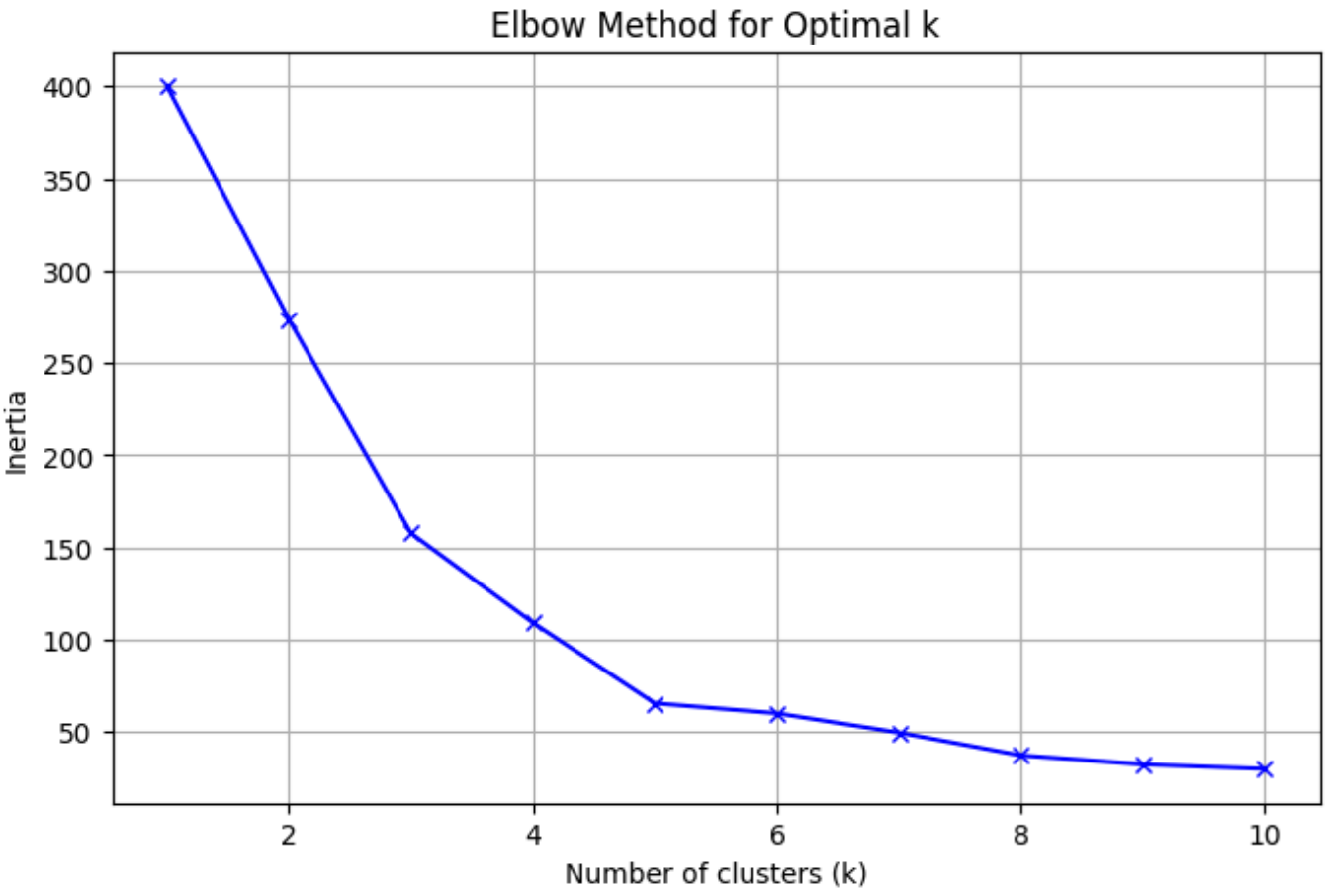
```
[-0.47940803,  0.34162973],
[-0.47940803, -0.00776431],
[-0.47940803, -0.08540743],
[-0.47940803,  0.34162973],
[-0.47940803, -0.12422899],
[-0.4412386 ,  0.18634349],
[-0.4412386 , -0.3183368 ],
[-0.40306917, -0.04658587],
[-0.40306917,  0.22516505],
[-0.25039146, -0.12422899],
[-0.25039146,  0.14752193],
[-0.25039146,  0.10870037],
[-0.25039146, -0.08540743],
[-0.25039146,  0.06987881],
[-0.25039146, -0.3183368 ],
[-0.25039146,  0.03105725],
[-0.25039146,  0.18634349],
[-0.25039146, -0.35715836],
[-0.25039146, -0.24069368],
[-0.25039146,  0.26398661],
[-0.25039146, -0.16305055],
[-0.13588317,  0.30280817],
[-0.13588317,  0.18634349],
[-0.09771374,  0.38045129],
[-0.09771374, -0.16305055],
[-0.05954431,  0.18634349],
[-0.05954431, -0.35715836],
[-0.02137488, -0.04658587],
[-0.02137488, -0.39597992],
[-0.02137488, -0.3183368 ],
[-0.02137488,  0.06987881],
[-0.02137488, -0.12422899],
[-0.02137488, -0.00776431],
[ 0.01679455, -0.3183368 ],
[ 0.01679455, -0.04658587],
[ 0.05496398, -0.35715836],
[ 0.05496398, -0.08540743],
[ 0.05496398,  0.34162973],
[ 0.05496398,  0.18634349],
[ 0.05496398,  0.22516505],
[ 0.05496398, -0.3183368 ],
[ 0.09313341, -0.00776431],
[ 0.09313341, -0.16305055],
[ 0.09313341, -0.27951524],
[ 0.09313341, -0.08540743],
[ 0.09313341,  0.06987881],
[ 0.09313341,  0.14752193],
[ 0.13130284, -0.3183368 ],
[ 0.13130284, -0.16305055],
[ 0.16947227, -0.08540743],
[ 0.16947227, -0.00776431],
[ 0.16947227, -0.27951524],
[ 0.16947227,  0.34162973],
[ 0.24581112, -0.27951524],
[ 0.24581112,  0.26398661],
[ 0.24581112,  0.22516505],
[ 0.24581112, -0.39597992],
[ 0.32214998,  0.30280817],
[ 0.32214998,  1.58391968],
[ 0.36031941, -0.82301709],
[ 0.36031941,  1.04041783],
[ 0.39848884, -0.59008772],
[ 0.39848884,  1.73920592],
[ 0.39848884, -1.52180518],
[ 0.39848884,  0.96277471],
[ 0.39848884, -1.5994483 ],
```

```
[ 0.39848884,  0.96277471],
[ 0.43665827, -0.62890928],
[ 0.43665827,  0.80748846],
[ 0.4748277 , -1.75473454],
[ 0.4748277 ,  1.46745499],
[ 0.4748277 , -1.67709142],
[ 0.4748277 ,  0.88513158],
[ 0.51299713, -1.56062674],
[ 0.51299713,  0.84631002],
[ 0.55116656, -1.75473454],
[ 0.55116656,  1.6615628 ],
[ 0.58933599, -0.39597992],
[ 0.58933599,  1.42863343],
[ 0.62750542, -1.48298362],
[ 0.62750542,  1.81684904],
[ 0.62750542, -0.55126616],
[ 0.62750542,  0.92395314],
[ 0.66567484, -1.09476801],
[ 0.66567484,  1.54509812],
[ 0.66567484, -1.28887582],
[ 0.66567484,  1.46745499],
[ 0.66567484, -1.17241113],
[ 0.66567484,  1.00159627],
[ 0.66567484, -1.32769738],
[ 0.66567484,  1.50627656],
[ 0.66567484, -1.91002079],
[ 0.66567484,  1.07923939],
[ 0.66567484, -1.91002079],
[ 0.66567484,  0.88513158],
[ 0.70384427, -0.59008772],
[ 0.70384427,  1.27334719],
[ 0.78018313, -1.75473454],
[ 0.78018313,  1.6615628 ],
[ 0.93286085, -0.93948177],
[ 0.93286085,  0.96277471],
[ 0.97103028, -1.17241113],
[ 0.97103028,  1.73920592],
[ 1.00919971, -0.90066021],
[ 1.00919971,  0.49691598],
[ 1.00919971, -1.44416206],
[ 1.00919971,  0.96277471],
[ 1.00919971, -1.56062674],
[ 1.00919971,  1.62274124],
[ 1.04736914, -1.44416206],
[ 1.04736914,  1.38981187],
[ 1.04736914, -1.36651894],
[ 1.04736914,  0.72984534],
[ 1.23821628, -1.4053405 ],
[ 1.23821628,  1.54509812],
[ 1.390894  , -0.7065524 ],
[ 1.390894  ,  1.38981187],
[ 1.42906343, -1.36651894],
[ 1.42906343,  1.46745499],
[ 1.46723286, -0.43480148],
[ 1.46723286,  1.81684904],
[ 1.54357172, -1.01712489],
[ 1.54357172,  0.69102378],
[ 1.61991057, -1.28887582],
[ 1.61991057,  1.35099031],
[ 1.61991057, -1.05594645],
[ 1.61991057,  0.72984534],
[ 2.00160487, -1.63826986],
[ 2.00160487,  1.58391968],
[ 2.26879087, -1.32769738],
[ 2.26879087,  1.11806095],
[ 2.49780745, -0.86183865],
```

```
       [ 2.49780745,  0.92395314],
       [ 2.91767117, -1.25005425],
       [ 2.91767117,  1.27334719]])
```

In [21]:
```python
# Determine optimal number of clusters using Elbow Method
inertias = []
K = range(1, 11)
for k in K:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_scaled)
    inertias.append(kmeans.inertia_)
```

In [22]:
```python
# Plot Elbow Curve
plt.figure(figsize=(8, 5))
plt.plot(K, inertias, 'bx-')
plt.xlabel('Number of clusters (k)')
plt.ylabel('Inertia')
plt.title('Elbow Method for Optimal k')
plt.grid(True)
plt.show()
```

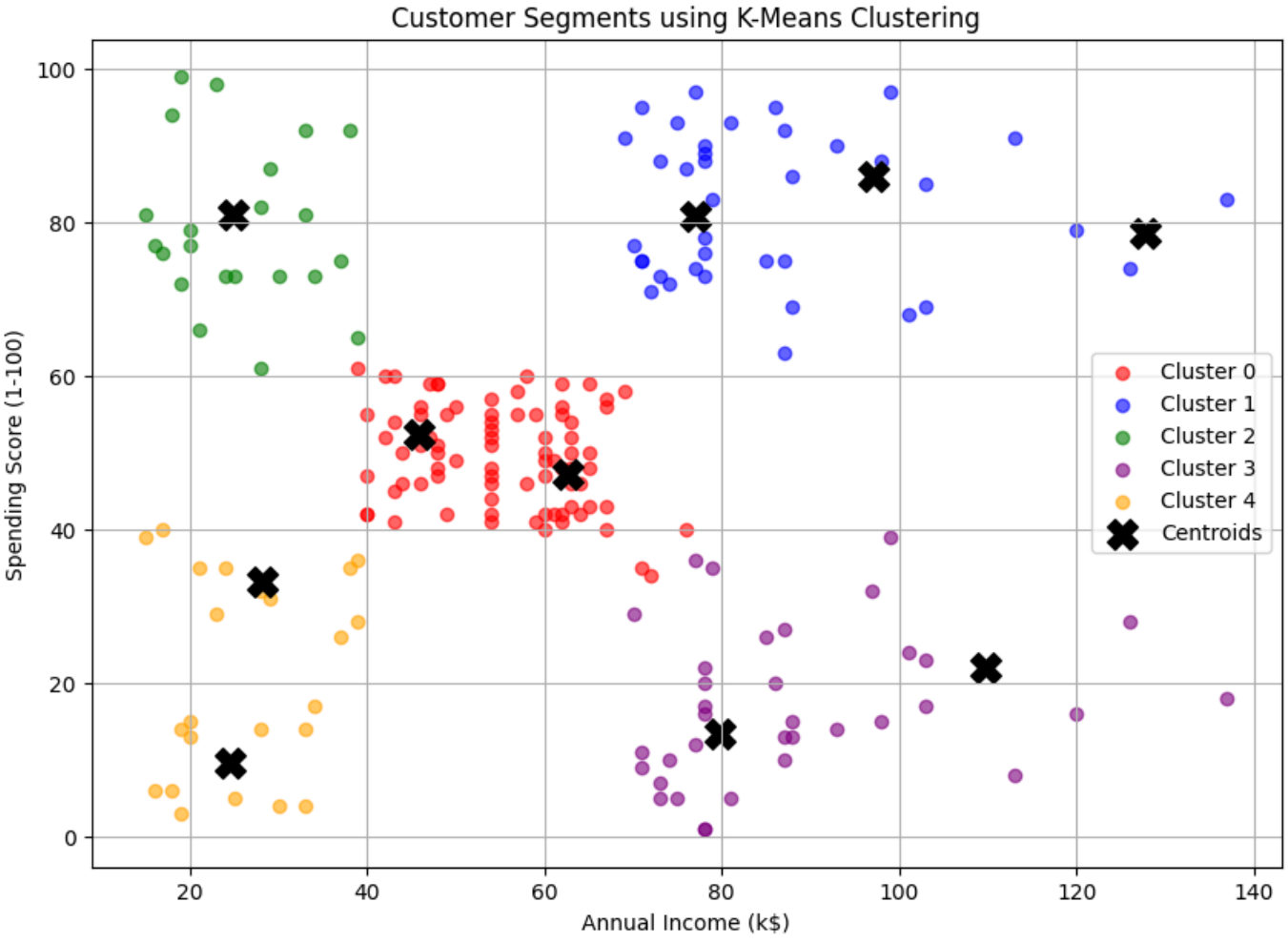


In [19]:
```python
# From elbow plot, optimal k = 5 (common for this dataset)
optimal_k = 5
kmeans = KMeans(n_clusters=optimal_k, random_state=42)
clusters = kmeans.fit_predict(X_scaled)
# Add cluster labels to original dataframe
df['Cluster'] = clusters
print("Cluster Centers (scaled):")
print(kmeans.cluster_centers_)
print("Cluster distribution:")
print(df['Cluster'].value_counts())
```

```
Cluster Centers (scaled):
[[-0.20091257 -0.02645617]
 [ 0.99158305  1.23950275]
 [-1.32954532  1.13217788]
 [ 1.05500302 -1.28443907]
 [-1.30751869 -1.13696536]]
Cluster distribution:
Cluster
0    81
1    39
3    35
4    23
2    22
Name: count, dtype: int64
```

In [23]:

```python
# Visualization of clusters
plt.figure(figsize=(10, 7))
colors = ['red', 'blue', 'green', 'purple', 'orange']
for i in range(optimal_k):
    plt.scatter(X[df['Cluster'] == i]['Annual Income (k$)'],
                X[df['Cluster'] == i]['Spending Score (1-100)'],
                label=f'Cluster {i}', c=colors[i], alpha=0.6)
# Plot centroids (inverse transform to original scale)
centroids = scaler.inverse_transform(kmeans.cluster_centers_)
plt.scatter(centroids[:, 0], centroids[:, 1], s=200, c='black', marker='X', label='Centroids'
plt.title('Customer Segments using K-Means Clustering')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.legend()
plt.grid(True)
plt.show()
```



In [24]:

```python
# Business Insights
print("=== Business Insights from Clusters ===")
print("Cluster 0: Medium income, medium spending → Standard customers")
```

```
print("Cluster 1: High income, high spending → Target/Premium customers")
print("Cluster 2: Low income, high spending → Carefree/Spenders")
print("Cluster 3: High income, low spending → Careful/Conservative customers")
print("Cluster 4: Low income, low spending → Sensible customers")
```

```
=== Business Insights from Clusters ===
Cluster 0: Medium income, medium spending → Standard customers
Cluster 1: High income, high spending → Target/Premium customers
Cluster 2: Low income, high spending → Carefree/Spenders
Cluster 3: High income, low spending → Careful/Conservative customers
Cluster 4: Low income, low spending → Sensible customers
```

In [ ]: